2025 IEEE 2nd International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS)
Bengaluru, India. Aug 29-30, 2025

2025 IEEE 2nd International Conference on Information Technology, Electronics and Intelligent Communication Systems (ICITEICS) | 979-8-3315-0784-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICITEICS64870.2025.11341738

Task Offloading in Edge Computing: Energy Optimization through SAC based Reinforcement Learning

Shanmukhi Lakshmi Sidwini
Dept. of Comp. Sc. & Information Systems
Birla Institute of Technology & Science, Pilani
Pilani, India
slsidwini@gmail.com

Suchismita Rout
School of Computer Engineering,
Kalinga Institute of Industrial Technology (KIIT) Deemed to be University
Bhubaneswar, India
suchismita.rout28@gmail.com

Harish Kumar Patnaik
School of Computer Engineering,
Kalinga Institute of Industrial Technology (KIIT) Deemed to be University
Bhubaneswar, India
hpatnaikfcs@kiit.ac.in

Bibhuti Bhusan Dash
School of Computer Applications
Kalinga Institute of Industrial Technology (KIIT) Deemed to be University
Bhubaneswar, India
bibhuti.dash@gmail.com

Utpal Chandra De
School of Computer Applications
Kalinga Institute of Industrial Technology (KIIT) Deemed to be University
Bhubaneswar, India
deutpal@gmail.com

Sudhansu Shekhar Patra
School of Computer Applications
Kalinga Institute of Industrial Technology (KIIT) Deemed to be University
Bhubaneswar, India
sudhanshupatra@gmail.com

***Abstract*— Task offloading is the process of transferring computationally intensive tasks from a local device to a remote resource, such as a server or cloud, to improve performance and efficiency. This paper focuses on minimizing energy consumption and delay in task offloading in cloud-edge systems. A Soft Actor-Critic (SAC) based reinforcement learning approach is proposed to dynamically decide where to offload computational tasks (local, edge, or cloud). A custom simulation environment was built using OpenAI Gym and Stable Baselines3. Comparative evaluations were conducted against DDPG and DQN. SAC demonstrated superior performance in balancing reward, energy efficiency, and delay, making it a promising solution for intelligent task offloading in edge computing.**

Keywords*— *Task offloading, Edge Computing, SAC, DQN, DDPG, Reinforcement Learning

I. INTRODUCTION

The rapid growth of Internet of Things (IoT) devices and the increasing demand for real-time and resource-intensive applications have led to significant challenges in computing infrastructure [1]. Traditional cloud computing, though powerful, often suffers from high latency and bandwidth limitations when dealing with latency-sensitive applications. To overcome these limitations, edge computing has emerged as a promising paradigm that brings computation closer to the data source, reducing response time and improving service quality [2]. However, managing the trade-off between computational performance, energy consumption, and task execution delay in a cloud-edge collaborative environment remains a complex problem. Task offloading, where computational tasks are dynamically offloaded to either edge servers or the cloud, plays a crucial role in addressing this challenge. The decision of whether to execute a task locally, at the edge, or in the cloud must consider several dynamic factors such as network conditions, device capabilities, and task characteristics. To optimize such decisions in a dynamic and uncertain environment, Reinforcement Learning (RL) and Deep Reinforcement Learning (DRL) techniques have gained significant attention. Unlike rule-based or heuristic approaches, RL-based models can learn optimal offloading strategies by interacting with the environment and maximizing long-term reward functions that account for energy efficiency, delay minimization, and resource utilization. In this work, We propose a DRL-based task offloading framework in a three-tier architecture comprising local devices, edge servers, and cloud data centers. We specifically implement and compare three prominent RL algorithms - Deep Q-Network (DQN), Deep Deterministic Policy Gradient (DDPG), and Soft Actor-Critic (SAC) [3,4] within a unified simulation environment. The proposed reward function balances energy consumption and task execution delay to achieve efficient and sustainable task offloading. Extensive simulations and performance evaluations are conducted to highlight the strengths and weaknesses of each approach [5,6].

The main contributions of this paper are as follows:

1) Comparison of three DRL algorithms (DQN, DDPG, SAC) for dynamic task offloading.
2) A comparative analysis of each model's performance based on training reward, average energy consumption, and delay.
3) Visualization of training and evaluation metrics to assess stability and convergence.

II. RELATED WORKS

Modern mobile applications often require fast computation and quick responses, which mobile devices alone cannot handle due to their limited processing power and battery. Task offloading allows mobile devices to send tasks to nearby edge servers or cloud data centres, reducing workload and saving energy while improving overall performance [7]. Earlier offloading strategies mainly used fixed rules or simple decision-making methods. These approaches considered things like task size or network bandwidth but couldn't adjust to changing conditions like user mobility or varying server loads [8,9] Reducing energy consumption is a major goal in task offloading [10], especially for mobile devices. Some studies focused on lowering both computation and communication energy costs. Many researchers aimed to minimize not just energy use but also delay. A few proposed optimization models that find a

balance between these two goals like DQN, DDPG, etc [11-13]. Although much progress has been made, many studies only focus on either energy consumption or delay but not both. Also, many existing methods don't handle real-time changes in network conditions or user behaviour. Future research should focus on dynamic, adaptive offloading strategies that consider multiple objectives and real-world variability [14-16]

III. SYSTEM MODEL

The overarching perspective of a three-tier hierarchical design comprises IoT and edge devices, edge servers, and cloud servers, as seen in Fig. 1. The first layer of the mobile edge architecture comprises edge devices that collect data from sensors. These devices are situated at the the network's periphery and facilitate user access. They may include smartphones, tablets, wearables, or any other IoT sensors linked to the network. Edge devices often possess inadequate processing power and storage space, rendering them unsuitable for executing complicated programs.A collection of edge servers is used in the second layer to handle incoming requests from edge (IoT) devices situated near the end user. These servers possess superior processing power and storage capacity relative to the edge devices. Additionally, they may transfer a portion of the incoming workload that cannot be processed locally to the subsequent layer in the design via the edge network. The tertiary layer of the mobile edge architecture comprises the cloud servers. The cloud offers universally accessible resources for the storage, processing, and analysis of large volumes of data. It may also provide many sorts of services.

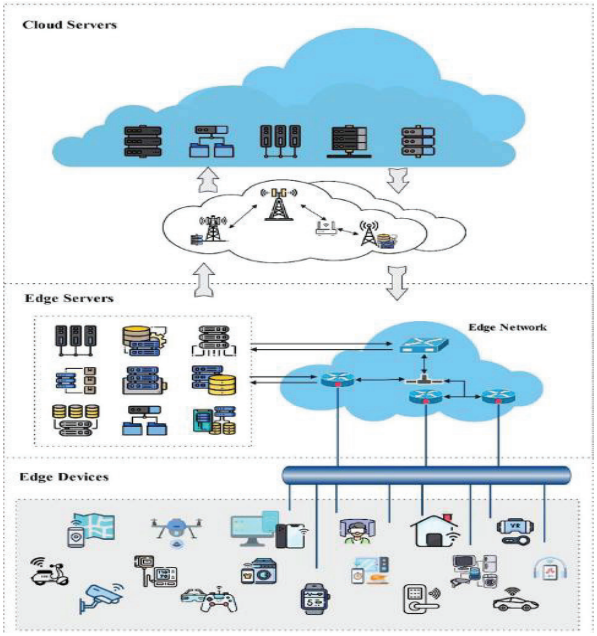

Fig. 1. Overview of the three-tier hierarchical edge computing architecture

The edge architecture has many advantages over traditional cloud computing platforms. It may diminish latency by situating computational and storage resources closer to the consumers. This is particularly vital for activities requiring instantaneous response, like as gaming or autonomous cars. Furthermore, it may alleviate network congestion by delegating some computational and storage demands to the edge servers. This may also improve the network's overall efficacy and dependability. Ultimately, it provides a cost-effective and scalable solution for the deployment of large-scale applications necessitating substantial computational and storage resources.

A. Mathematical Equations Related to Task Offloading

1) TASK

$$T = \{D, C\} \tag{1}$$

Where, T : is the Task, D: Data size of the Task and C: Required CPU cycles per bit(in cycles/bit)

2) EXECUTION TIME (DELAY)

As we are taking a three-tier system:

Here is the Execution delay of these three during processing:-

a) LOCAL EXECUTION TIME

$$t_{\text{local}} = \frac{D \cdot C}{f_{\text{local}}} \tag{2}$$

Where , f_{local} : is the CPU frequency of the local device(in cycles/sec)

b) EXECUTION TIME IN EDGE

$$t_{\text{edge}} = t_{\text{tx}}^{\text{edge}} + \frac{D \cdot C}{f_{\text{edge}}} \tag{3}$$

Where, $t_{\text{tx}}^{\text{edge}} = \frac{D}{r_{\text{edge}}}$: Transmission Time to edge,

r_{edge} : Uplink data rate (bps), f_{edge} : Frequency in Edge server

c) EXECUTION TIME IN CLOUD

$$t_{\text{cloud}} = t_{\text{tx}}^{\text{cloud}} + \frac{D \cdot C}{f_{\text{cloud}}} \tag{4}$$

Where, $t_{\text{tx}}^{\text{cloud}} = \frac{D}{r_{\text{cloud}}}$: Transmission Time to Cloud, : Uplink data rate (bps), f_{cloud} : Frequency in Edge server. It is similar to Edge Execution Time Equation (3), but with different data rate and frequency.

3) ENERGY CONSUMPTION

Here we have two types of Energy Consumptions as follows:-

a) LOCAL ENERGY

$$E_{\text{local}} = \kappa \cdot C \cdot D \cdot f_{\text{local}}^2 \tag{5}$$

Where is a constant which is depended on the device hardware.

b) TRANSMISSION ENEGRY

Here we are discussing on the transmission energy during the offloading to edge server or cloud server.

$$E_{\text{tx}}^{\text{edge/cloud}} = P_{\text{tx}} \cdot t_{\text{tx}}^{\text{edge/cloud}} \tag{6}$$

Where, P_{tx} : represents the power consumed by the device's transmitter when it is sending data to another computing unit with either an edge server or a cloud server.

4) REWARD

The reward function guides the RL agent toward better behaviour. In Better points:-

- Minimize Energy → Save battery life
- Minimize Delay → Ensure fast execution

$$R = -(\alpha \cdot E_{\text{total}} + \beta \cdot t_{\text{total}}) \quad (7)$$

Where, α and β : are the weight factors to control the trade-off, E_{total} : is the total energy consumed and t_{total} : is the total time taken. Here in the given equation, it is negative, so that RL agent learns to minimize the cost. Over time, the agent learns to pick actions that reduce energy and delay. These are the core building blocks or equations which are necessary during the Studying or implementation of Task offloading. These help to understand - How much a task takes to be completed during the task offloading in short execution time and How much energy is consumed to complete a task whether its run locally, on edge or cloud server. How the network impacts performance, as the network condition directly affects transmission time and transmission energy, which in turn affect overall delay and energy cost.

IV. SOFT ACTOR CRITIC RL

SAC is an advanced reinforcement learning algorithm designed for continuous action spaces. It uses an actor-critic architecture where the actor learns a stochastic policy (choosing actions with some randomness), and two critic networks evaluate the actions to reduce overestimation errors.

What makes SAC unique is that it adds entropy to the objective, encouraging the agent to explore more instead of settling too quickly on suboptimal actions. This results in better stability, improved performance, and greater exploration in complex environments.SAC also uses experience replay and target networks for stability and supports automatic entropy tuning, which simplifies hyperparameter tuning and makes it more adaptive to different tasks.

Key Contributions of SAC: -

Entropy-Regularized Learning

- SAC doesn't just try to maximize reward, but also maximize entropy (i.e., randomness of actions).
- Which gives more exploration which leads to less chance of getting stuck in suboptimal policies.
- SAC also strike a great balance between exploration and exploitation.

Stochastic Actor-Critic Architecture

- SAC also uses two neural networks:
  - Actor: Outputs a probability distribution over actions (not just one fixed action like DDPG).
  - Critic(s): Estimates how good actions are, where usually two Q-networks to reduce overestimation (called Double Q-learning).
- This allows robust decision making, especially in uncertain or dynamic environments.

Works in Continuous Action Spaces (and Stochastic)

- Like DDPG, SAC operates in continuous action domains, which is essential for real-world control tasks like:
  - Robotics
  - Task Offloading
  - Edge-Cloud Environments

Automatic Temperature Tuning

- SAC introduces a temperature parameter ( ) to weigh reward vs. entropy.
- It can automatically tune this parameter which removes the need for manual tweaking which results to more stable and adaptive learning.

Experience Replay + Target Networks

- Reuses past experiences using a replay buffer.
- Maintains target networks for the critics (slowly updated) which ensures stable learning and avoids drastic policy changes.

i. Soft Q-function Loss (Critic Update)

$$\mathrm{L}_Q(\theta_i) = \mathrm{E}_{(s_t, a_t, r_t, s_{t+1})\sim \mathrm{D}} \left[\left(Q_{\theta_i}(s_t, a_t) - y_t \right)^2 \right] \quad (8)$$

TABLE I.

Symbol	Meaning
$\mathrm{L}_Q(\theta_i)$	Critic loss function for the i-th Q-function
$\mathrm{E}_{(s_t, a_t, r_t, s_{t+1})\sim \mathrm{D}}$	Expectation over sampled transitions from the replay buffer D
$Q_{\theta_i}(s_t, a_t)$	Estimated Q-value by the i-th critic network for state-action pair
y_t	Target Q-value computed using next state and action (see next equation)

ii. Target for Q-function Loss

$$y_t = r_t + \gamma \mathrm{E}_{a_{t+1}\sim \pi_\phi} X$$

$$\left[\min_{j=1,2} Q_{\bar{\theta}_j}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\phi(a_{t+1} | \, s_{t+1}) \right] (9)$$

TABLE II.

Symbol	Meaning
y_t	Target Q-value used in critic update
r_t	Reward at time step t
γ	Discount factor for future rewards
$a_{t+1}\sim \pi_\phi$	Action sampled from the current policy at next state
$Q_{\bar{\theta}_j}(s_{t+1}, a_{t+1})$	Target Q-function with delayed parameters $Q_{\bar{\theta}_j}$
$\min_{j=1,2}$	Minimum over two Q-value estimates to reduce overestimation bias
$\alpha \log \pi_\phi(a_{t+1} \mid s_{t+1})$	Entropy regularization term weighted by temperature α

iii. Policy Loss (Actor Update)

$$L_{\pi}(\phi) = E_{s_t \sim D}\left[E_{a_t \sim \pi_\phi}\left[\alpha \log \pi_\phi(a_t \mid s_t) - \min_{i=1,2} Q_{\theta_i}(s_t, a_t) \right] \right] \quad (10)$$

TABLE III.

Symbol	Meaning
$L_{\pi}(\phi)$	Loss function for the policy (actor) network with parameters ϕ
$s_t \sim D$	State sampled from replay buffer
$a_t \sim \pi_\phi$	Action sampled from policy π_ϕ
$\alpha \log \pi_\phi(a_t \mid s_t)$	Entropy term encouraging exploration
$Q_{\theta_i}(s_t, a_t)$	Critic Q-value estimate for action a_t in state s_t
$\min_{i=1,2}$	Minimum across both critics for stability

iv. Temperature (Entropy) Loss

$$L_{\alpha}(\alpha) = E_{a_t \sim \pi_\phi}\left[-\alpha\left(\log \pi_\phi(a_t \mid s_t) + H_{\text{target}} \right) \right] \quad (11)$$

TABLE IV.

Symbol	Meaning
L_{α}	Loss function for adjusting entropy temperature
α	Temperature coefficient controlling entropy weight
$\pi_\phi(a_t \mid s_t)$	Probability of action under current policy
H_{target}	Target entropy

v. Target Network Update (Polyak Averaging)

$$\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i, \quad \text{for } i = 1,2 \quad (12)$$

TABLE V.

Symbol	Meaning
$\bar{\theta}$	Parameters of the target Q-function (soft-updated)
θ_i	Parameters of the current Q-function
τ	Polyak averaging coefficient (e.g., 0.005)
$i = 1,2$	Index for the two Q-networks

Below are the points of how SAC addresses the key limitations found in DQN and DDPG:

Soft Actor-Critic vs. DQN & DDPG – Improvements: -

1. Twin Q-Functions (Q_1 and Q_2):

- As discussed earlier, In DDPG there was a drawback of Overestimates Q-values due to using a single critic and in DQN, Max operator in target (from equation 14) introduces overestimation bias.
- But in SAC it uses two Q-functions and takes min(Q_1, Q_2) during updates to reduce overestimation (like TD3) which leads to more conservative and stable value estimates.

2. Stochastic Policy Learning $\pi_\phi(a_t \mid s_t)$:

- In DDPG it uses a deterministic policy which leads to poor exploration.
- In DQN It doesn't learn a policy at all, it just uses ε-greedy action selection.
- But in SAC it learns a stochastic policy $\pi_\phi(a_t \mid s_t)$, which encourages natural exploration through sampling which prevents policy from collapsing to suboptimal greedy behaviour.

3. Entropy-Regularized Objective:

- In Both DDPG/DQN a fixed exploration strategy (noise or ε) is used due to which it cannot learn or state-sensitive.
- In SAC, it adds entropy term ($\log \pi_\phi$) to reward, encouraging diverse action selection. This enables better exploration, especially in sparse-reward or deceptive environments.

4. Learnable Temperature Parameter :

- In DDPG/DQN there is no way to adapt exploration vs. exploitation trade-off.
- But in SAC α, it learns to automatically adjust how much entropy to encourage. Which helps tune the policy to be more exploratory or focused depending on the learning stage.

5. Target Value with Entropy:

- In DDPG, Target is deterministic and prone to bias.
- In DQN Limitation it uses hard max which can inflate Q-values.
- In SAC, Target is:

$$y_t = r_t + \gamma E_{a_{t+1} \sim \pi_\phi} X \quad (13)$$
$$\left[\min_{j=1,2} Q_{\bar{\theta}_j}(s_{t+1}, a_{t+1}) - \alpha \log \pi_\phi(a_{t+1} \mid s_{t+1}) \right]$$

which produces a softened, lower-variance, entropy-regularized target that encourages exploration.

6. Soft Target Updates (Polyak Averaging):

- In DDPG/DQN we observe a sharp target update can destabilize learning.
- In SAC we can observe a slow update target Q-networks: $\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i, \quad \text{for } i = 1,2$ (14)

which gives a smooth update and keep learning stable and consistent.

IV. RESULTS ANALYSIS

This section, we present the evaluation results of the three reinforcement learning algorithms Soft Actor-Critic (SAC), Deep Deterministic Policy Gradient (DDPG), and Deep Q-Network (DQN) in the context of the task offloading problem in cloud-edge environments. The primary evaluation metrics used for comparison include average reward, average energy consumption, and average delay.

A. Evaluation Metrics

The code implements a reinforcement learning setup for task offloading in a multi-tier system, using the SAC, DDPG, and DQN algorithms from Stable-Baselines3. It defines a custom Gym environment (TaskOffloadingEnv) where an agent decides on task offloading strategies between local devices, edge devices, and cloud servers. The environment simulates task size, CPU cycles, bandwidth, energy consumption, and delay. Discrete and continuous action spaces are handled for DQN and SAC/DDPG respectively, with offloading fractions as actions. The models are trained for 50,000 timesteps each, and their performance is evaluated through metrics like reward, energy consumption, and delay. The results saved to CSV files and are compared using plots for analysis.

1. Average Reward: Indicates the overall efficiency of the task offloading process, balancing energy consumption, execution delay, and resource utilization. A higher (less negative) reward indicates better performance.
2. Average Energy Consumption: Tracks the energy consumed by the system during task offloading. Reducing energy consumption is critical for sustainability and efficiency.
3. Average Delay: Reflects the total time taken for tasks to be offloaded and completed. Lower delay means faster execution and better system responsiveness.

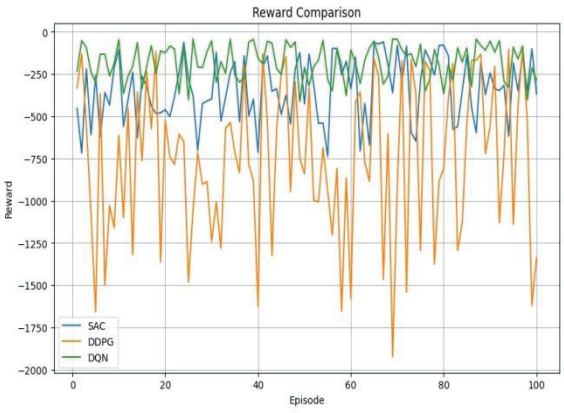

Fig. 2. Comparison Plot for Reward between SAC vs. DQN vs. DDPG

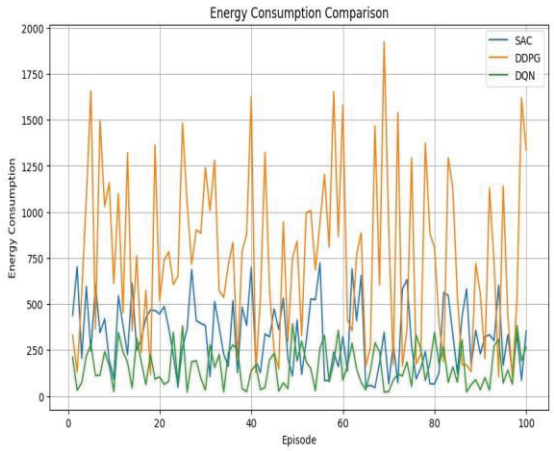

Fig. 3. Comparison Plot for Energy Consumption between SAC vs. DQN vs. DDPG

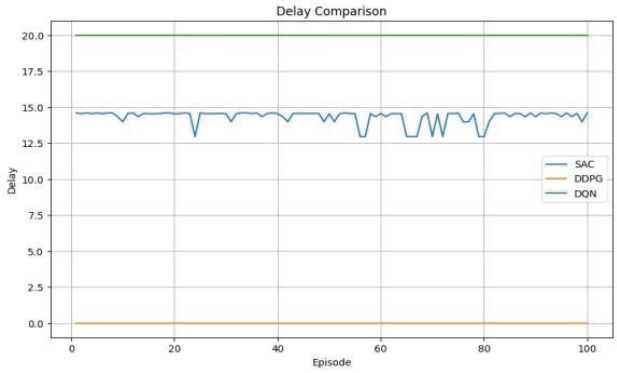

Fig. 4. Comparison Plot for Delay between SAC vs. DQN vs. DDPG

SAC outperforms both DDPG and DQN as shown in figs 2 to 4 by maintaining a solid balance between energy consumption and delay, resulting in a more optimized reward. DDPG, despite achieving zero delay, is highly inefficient in energy consumption, making it unsuitable for low-power edge environments.DQN performs well in energy consumption but suffers from long delays, indicating room for improvement in task scheduling and resource allocation strategies.

V. CONCLUSION & FURUTE WORK

SAC emerges as the most suitable algorithm for task offloading in cloud-edge environments. Its ability to optimize both energy and delay while achieving a favourable reward makes it ideal for dynamic, resource-constrained systems. DDPG's focus on minimizing delay at the expense of energy is impractical in real-world scenarios. Meanwhile, DQN, although energy-efficient, fails to reduce task delay, limiting its effectiveness. These findings reinforce SAC's robustness and adaptability, making it a strong candidate for future real-world deployment and research in energy-aware task offloading strategies. As task offloading environments can sometimes be unstable in rewards (especially when the system is in early stages of training), improving exploration strategies (such as randomized exploration) in SAC could help the agent find better solutions more quickly. The reward function used in SAC can be dynamically adjusted based on real-time environmental conditions such as network fluctuations, task priority, or energy availability. This could lead to more responsive task offloading decisions that better balance energy and delay in different situations. SAC can be extended to a multi-agent system like DRL, where each device in the edge network is treated as an agent could lead

to better resource allocation strategies. Agents could learn how to cooperate and share resources for optimal task offloading in a large-scale system.

REFERENCES

[1] L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, & L. Li,." Delay-aware and energy-efficient computation offloading in mobile-edge computing using deep reinforcement learning," IEEE Transactions on Cognitive Communications and Networking, 7(3), pp. 881-892, 2021

[2] Z. Li, V. Chang, J. Ge, L. Pan, H. Hu, & B. Huang, "Energy-aware task offloading with deadline constraint in mobile edge computing," EURASIP Journal on Wireless Communications and Networking, pp. 1-24, 2021,.

[3] Z. Zabihi, A.M. Eftekhari Moghadam, & M.H. Rezvani, "Reinforcement learning methods for computation offloading: a systematic review," ACM Computing Surveys, 56(1), pp.1-41, 2023.

[4] Y. Hou, L. Liu, Q. Wei, X. Xu, & C. Chen, " A novel DDPG method with prioritized experience replay," In 2017 IEEE international conference on systems, man, and cybernetics (SMC) , pp. 316-321). IEEE, 2017.

[5] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M.G. Bellemare, ... & D. Hassabis, "Human-level control through deep reinforcement learning," nature, 518(7540), 529-533, 2015.

[6] F. Zhang, G. Han, L. Liu, M. Martínez-García, & Y. Peng, "Deep reinforcement learning based cooperative partial task offloading and resource allocation for IIoT applications," IEEE Transactions on Network Science and Engineering, 10(5), 2991-3006, 2022.

[7] M. Tang, & V. W. Wong, " Deep reinforcement learning for task offloading in mobile edge computing systems, "IEEE Transactions on Mobile Computing, 21(6), pp. 1985-1997, 2020.

[8] E.H. Sumiea, S. J. Abdulkadir, H. S. Alhussian, S.M. Al-Selwi, A. Alqushaibi, M.G. Ragab, & S.M. Fati, "Deep deterministic policy gradient algorithm: A systematic review," Heliyon, 2024.

[9] T. P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, ... & D. Wierstra, "Continuous control with deep reinforcement learning," arXiv preprint arXiv:1509.02971, 2015.

[10] D. Karunakaran, "Deep Deterministic Policy Gradient (DDPG)—an off-policy Reinforcement Learning algorithm," Intro to Artificial Intelligence, 25, 2020.

[11] J. Hui, "RL — DQN Deep Q-network," Medium, Jul. 2018. [Online]. Available: https://jonathan-hui.medium.com/rl-dqn-deep-q-network-e207751f7ae4 Medium

[12] W. Hou, H. Wen, H. Song, W. Lei, and W. Zhang, "Multi-agent deep reinforcement learning for task offloading and resource allocation in Cybertwin-based networks," ResearchGate, 2021. [Online]. Available: https://www.researchgate.net/publication/353113391_Multi-Agent_Deep_Reinforcement_Learning_for_Task_Offloading_and_Resource_Allocation_in_Cybertwin_based_Networks

[13] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," arXiv preprint arXiv:1801.01290, [Online]. Available: https://arxiv.org/abs/1801.01290, 2018

[14] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with Deep Reinforcement Learning," arXiv preprint arXiv:1312.5602, 2013.

[15] T. Wang, Y. Deng, Z. Yang, Y. Wang, & H. Cai, "Parameterized deep reinforcement learning with hybrid action space for edge task offloading," IEEE Internet of Things Journal, 11(6), pp. 10754-10767, 2023.

[16] Z. Yang, Y. Deng, T. Wang, & H. Cai, "Towards efficient task offloading at the edge based on meta-reinforcement learning with hybrid action space. In ICC 2023-IEEE International Conference on Communications, pp. 4039-4044). IEEE, 2023.